Agentic AI-Based Intelligent Travel Planning System

Software Requirement Specification

Final-Year Project (BTech Information Technology)
Academic Prototype

Document Issue Status

Document ID: ATP/SRS		Document Issue Number: 1 Document Revision Number: 0	
Prepared by	Ashfaque C , Muhammad Fawas K Abel Mathew , Afrah Shad EK	Prepared Date:	
Reviewed by		Reviewed Date:	
Approved by		Approved Date:	
Distribution list			

Document Amendment Record

Sl. No.	Issue No	Revision No	Date	Pages Corrected	Remarks
1	1	0		All	Initial issue

Contents

1 Introduction 5
1.1 Purpose 5
1.2 Scope 5
1.3 Product / System Overview 5
1.4 User Characteristics 6
1.5 Definitions, Acronyms & Abbreviations 6
1.6 Constraints 7
1.7 Assumptions and Dependencies 7
1.8 References 7
2 Functional Requirements 8
2.1 Travel Request Module 8
2.2 Supervisor Agent 8
2.3 Transit Agent 8
2.4 Lodging Agent 9
2.5 Geospatial Agent 9
2.6 Safety Agent 9
2.7 Budget Management 9
2.8 Itinerary Generation 9
2.9 Dynamic Replanning 10
2.10 Map and Visualization 10
2.11 Data Storage 10
2.12 Use Cases 10
3 Non-Functional Requirements 12
3.1 Performance Requirements 12
3.2 Security Requirements 12
3.3 Software System Attributes 12
4 External Interface Requirements 14
4.1 User Interfaces 14
4.2 Hardware Interfaces 14
4.3 Software Interfaces 14
4.4 Communication Interfaces 14
5 Acceptance Criteria 15
6 Appendices 15
6.1 User Need Document 15
6.2 Requirement Traceability Matrix 15
6.3 Product Backlog 16
6.4 Future Enhancements 16

1 Introduction

1.1 Purpose

This Software Requirement Specification (SRS) defines the functional and non-functional requirements of the Agentic AI-Based Intelligent Travel Planning System. It describes what the system must do so that designers can design it and testers can test it.

Intended audience: the project team (developers and testers), the project guide and review panel, and students or researchers who want to study the system as an example of agentic AI and multi-agent coordination.

1.2 Scope

The software product is called the Agentic AI-Based Intelligent Travel Planning System. It is a web application that turns a natural-language travel request into a personalized, budget-aware, day-wise itinerary. A Supervisor Agent coordinates four specialized agents: Transit, Lodging, Geospatial and Safety.

The system will:

- Accept a travel request (origin, destination, duration, travelers, budget, preferences, interests).
- Suggest transport, accommodation, attractions and restaurants, and estimate routes, distances and costs.
- Check weather and nearby essential services (hospitals, pharmacies, police stations).
- Check the plan against budget, duration and preferences, and replan when a constraint fails or something changes.
- Show the day-wise itinerary, estimated budget and an interactive map.

The system will not:

- Book tickets or hotels, or process payments.
- Guarantee exact fares, availability or real-time safety alerts. All costs and recommendations are estimates unless a verified source supplies current data.

Benefit and goal: reduce the manual effort of searching many separate platforms, using a modular, low-cost, open-source approach. The prototype uses open-source tools and public services where possible. This matches the project summary.

1.3 Product / System Overview

The system has a React frontend, a FastAPI backend, a LangGraph-based multi-agent layer (Supervisor plus four agents) powered by a local LLM through Ollama, and a PostgreSQL/PostGIS database. Agents get data from tools and data sources (OpenStreetMap services, OSRM, Open-Meteo, transit feeds and a curated accommodation dataset). The tools are not agents.

Workflow: the user submits a request, the Supervisor splits it into tasks, the four agents fetch results, the Supervisor validates budget and constraints, and if constraints are not satisfied it replans. Otherwise it produces the itinerary, map and budget summary.

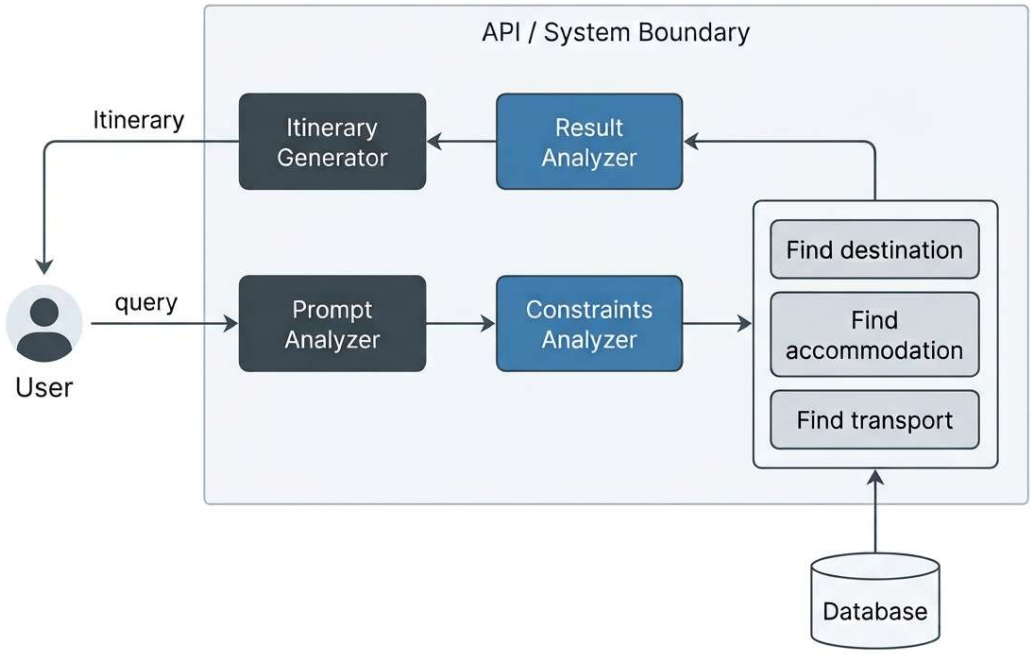


Figure 1: System block diagram

1.4 User Characteristics

User	Characteristics and effect on requirements
Traveler	Basic computer and internet knowledge. Does not know about AI, LLMs, APIs or databases. So the interface must be simple and accept plain-language requests (NFR-16, NFR-17).
Student / Researcher	Technical background. Uses the system to study agentic AI, multi-agent systems and geospatial planning. Needs clear separation of agents and tools (NFR-23, NFR-24).
Administrator	Technical user who maintains configuration, datasets and accommodation records.

1.5 Definitions, Acronyms & Abbreviations

Term	Meaning
Agent	A task-oriented software component that does one specific job
Agentic AI	AI that can plan, reason, use tools, coordinate actions and adapt to change
LLM	Large Language Model
LangGraph	Framework used to orchestrate the agent workflow and shared state
Ollama	Tool for running an open-source LLM locally
API	Application Programming Interface
POI	Point of Interest (attraction, restaurant, hospital, etc.)
OSM	OpenStreetMap
OSRM	Open Source Routing Machine (road routes and distances)

Term	Meaning
OTP	OpenTripPlanner (multimodal routing)
GTFS	General Transit Feed Specification (public transit data)
PostGIS	Geographic extension for PostgreSQL
SRS / UI / UX	Software Requirement Specification / User Interface / User Experience

1.6 Constraints

- Data and regulatory: recommendation quality depends on external data. Public APIs (Nominatim, Overpass, OSRM, Open-Meteo) have rate limits and usage policies that must be respected.
- Data coverage: public transport data (GTFS) may not exist for every location. Real-time hotel availability needs commercial or partner APIs, so the prototype uses a curated accommodation dataset.
- Hardware: a local LLM through Ollama needs enough RAM/GPU; model size and speed limit response time and quality.
- Language and tools: Python backend (FastAPI, LangGraph) and React frontend; PostgreSQL with PostGIS for data.
- Reliability: LLM output may contain errors and must not be treated as the authority for prices, schedules, ratings or safety.
- Security: API credentials must stay on the backend and user data must be protected.
- Criticality: the system is an academic prototype, not a commercial booking platform. Safety information is advisory only.

1.7 Assumptions and Dependencies

- Users give reasonably valid travel requirements.
- External services (Nominatim, Overpass, OSRM, Open-Meteo) are reachable during planning.
- Map data exists for the selected destination.
- Transit feeds are available wherever transit recommendations are needed.
- Accommodation data is available from the local dataset or an approved provider.
- The chosen local LLM is capable enough for requirement extraction and itinerary text.
- Windows or Linux with Python, Node.js, Docker (where needed), PostgreSQL with PostGIS and Ollama installed.

1.8 References

No.	Document	Source
1	Agentic AI-Based Intelligent Travel Planning System: Professional Project Summary	Project team
2	Software Requirement Specification Template	Institution / course
3	Documentation of LangGraph, FastAPI, Ollama, PostgreSQL/PostGIS, MapLibre GL JS	Official project websites
4	Documentation of Nominatim, Overpass API, OSRM, Open-Meteo, GTFS, OpenTripPlanner	Official project websites

2 Functional Requirements

Each requirement has a unique ID. Requirements are written so that users or external systems can observe them. Input, output and processing are listed per module.

2.1 Travel Request Module

Input: origin, destination, duration, travelers, budget, preferences, interests, or a free-text request. Output: a validated travel request passed to the Supervisor Agent.

ID	Requirement
FR-01	The system shall let the user enter the starting location.
FR-02	The system shall let the user enter the destination.
FR-03	The system shall let the user specify travel duration.
FR-04	The system shall let the user specify the number of travelers.
FR-05	The system shall let the user specify the total budget.
FR-06	The system shall let the user specify transport preferences.
FR-07	The system shall let the user specify accommodation preferences.
FR-08	The system shall let the user specify interests (historical places, nature, shopping, food, entertainment).
FR-09	The system shall accept natural-language travel requests.
FR-10	The system shall validate mandatory information before planning starts.

2.2 Supervisor Agent

Interprets the request, delegates to the specialized agents, validates results and produces the final plan.

ID	Requirement
FR-11	Interpret the travel request and extract parameters from natural-language input (FR-12).
FR-12	Extract origin, destination, duration, travelers, budget, interests and preferences.
FR-13	Decompose the request into subtasks.
FR-14	Delegate transport tasks to the Transit Agent.
FR-15	Delegate accommodation tasks to the Lodging Agent.
FR-16	Delegate location and route tasks to the Geospatial Agent.
FR-17	Delegate weather and safety tasks to the Safety Agent.
FR-18	Collect and validate results from all agents.
FR-19	Check the plan against budget, duration, route practicality and preferences ("Constraints satisfied?").
FR-20	Calculate or coordinate the total estimated expenditure.
FR-21	Generate the final day-wise itinerary when constraints are satisfied.
FR-22	Start replanning or additional searches when constraints are not satisfied or changes are detected.

2.3 Transit Agent

ID	Requirement
FR-23	Identify suitable transport options where data is available (OSRM, GTFS, OTP).

ID	Requirement
FR-24	Consider the user's transport preferences.
FR-25	Obtain route information, estimated distance and estimated travel time.
FR-26	Estimate transport cost when suitable data is available.
FR-27	Compare transport alternatives and return suitable options to the Supervisor.

2.4 Lodging Agent

ID	Requirement
FR-28	Identify accommodation options from the curated dataset in PostgreSQL.
FR-29	Filter options by the user's budget, location, rating (where available) and preferences.
FR-30	Estimate accommodation expenses and return suitable options to the Supervisor.

2.5 Geospatial Agent

ID	Requirement
FR-31	Convert place names into coordinates (Nominatim).
FR-32	Identify tourist attractions, restaurants and cafés (Overpass/OSM).
FR-33	Identify nearby points of interest and support distance-based filtering (PostGIS).
FR-34	Calculate distances and obtain road routes (OSRM).
FR-35	Return location and route information to the Supervisor.

2.6 Safety Agent

ID	Requirement
FR-36	Obtain weather information for planned destinations (Open-Meteo).
FR-37	Identify unfavorable weather and send warnings to the Supervisor.
FR-38	Identify mapped hospitals, police stations and pharmacies near the destination where data exists.
FR-39	Notify the Supervisor when safety conditions may affect the itinerary.

2.7 Budget Management

Expenses are divided into Transportation, Accommodation, Food, Activities and Other.

ID	Requirement
FR-40	Calculate estimated trip expenses per category.
FR-41	Compare the estimate against the user's budget and identify violations.
FR-42	Try to find lower-cost alternatives when the estimate exceeds the budget.
FR-43	Display the estimated total cost and remaining budget.

2.8 Itinerary Generation

ID	Requirement
FR-44	Generate a day-wise itinerary with planned activities, transport information and estimated costs.
FR-45	Consider geographic proximity between activities, available time and user preferences.

ID	Requirement
FR-46	Present results as an interactive map and budget summary.

2.9 Dynamic Replanning

Replanning is triggered by heavy rain, safety warnings, budget changes, unavailable transport or accommodation, excessive travel time or changed preferences.

ID	Requirement
FR-47	Monitor relevant information during planning and identify conflicts affecting the itinerary.
FR-48	Notify the Supervisor, which re-evaluates the affected itinerary parts.
FR-49	Request alternatives from the specialized agents when required.
FR-50	Generate an updated itinerary.

2.10 Map and Visualization

ID	Requirement
FR-51	Display destination, attractions, accommodation, restaurants and other points of interest on an interactive map (MapLibre).
FR-52	Display routes where route data is available and show the relationship between planned locations.

2.11 Data Storage

Main entities: User, Trip, Destination, Accommodation, Attraction, Restaurant, Transportation, Weather, Itinerary, Budget, AgentResult.

ID	Requirement
FR-53	Store travel requests, accommodation data, geographic data and generated itineraries in PostgreSQL.
FR-54	Retrieve stored information during planning.
FR-55	Support coordinates and spatial queries, for example "restaurants within 2 km of the selected hotel" (PostGIS).

2.12 Use Cases

Figure 2 shows the actors and use cases of the system. The Traveler creates, modifies and views travel plans. Creating a plan always includes the work of the Supervisor and the four specialized agents. Replanning extends it only when a constraint fails or conditions change. The Administrator maintains the hotel data and system settings. External services supply transit, map and weather data.

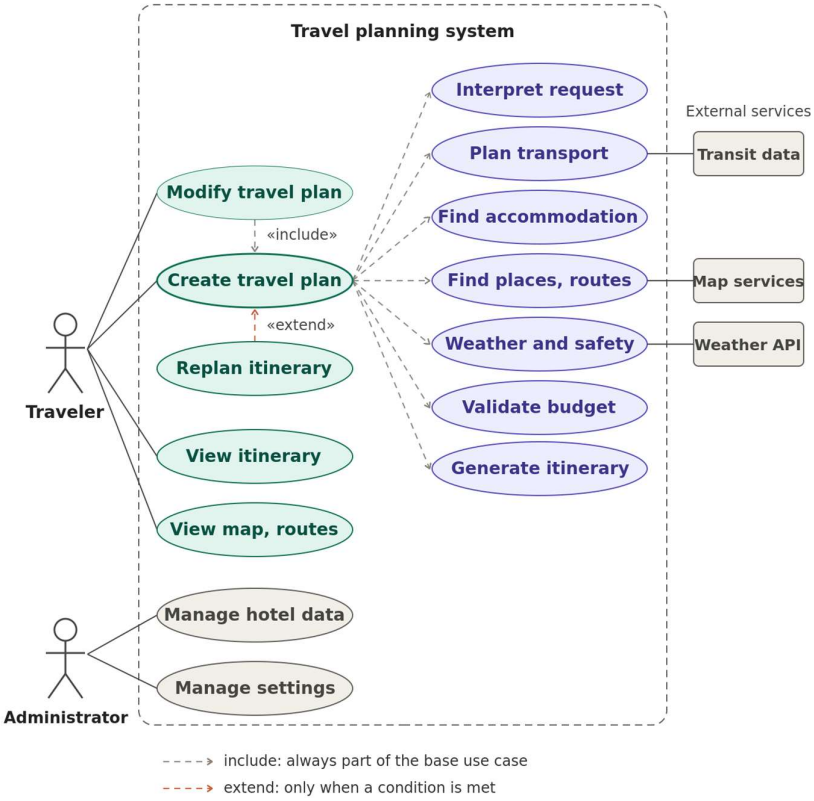


Figure 2: Use case diagram

UC-01: Create Travel Plan (Actor: Traveler)

- Precondition: user can open the web application.
- Flow: user enters origin, destination, duration, travelers, budget and preferences, then submits. The backend validates the request. The Supervisor interprets it and delegates to the four agents. Results return to the Supervisor, which validates them and checks the budget. If constraints are satisfied, the itinerary, map and budget summary are shown.
- Alternate flow: if constraints are not satisfied, the Supervisor replans (more searches or lower-cost options) and checks again.

UC-02: Modify Travel Plan (Actor: Traveler)

- Flow: user selects an existing plan and changes a requirement. The Supervisor re-evaluates, calls the relevant agents, and a revised itinerary is generated.

UC-03: Dynamic Replanning (Actor: System)

- Trigger: the Safety Agent detects an unfavorable condition, such as heavy rain.
- Flow: the Safety Agent informs the Supervisor, which identifies the affected activities. The Geospatial Agent finds alternatives. The system checks route, time, cost and preferences, then updates the itinerary.

Example scenario

Input: "Plan a 5-day trip from Kochi to Hyderabad for 2 people with a budget of ₹6000. Prefer budget transportation and accommodation. I am interested in historical places and local food." Expected output: transport options, stays, attractions, local food places, routes, weather and safety information, daily and total estimated cost, remaining budget, day-wise itinerary and map. The ₹6000 is a user-defined constraint, not a guarantee that the trip is feasible at that price.

3 Non-Functional Requirements

3.1 Performance Requirements

The targets below are prototype goals on a typical development laptop running a local LLM, and will be confirmed during testing.

ID	Requirement
NFR-01	The system shall return a complete itinerary for a typical request within 2 minutes in at least 90% of runs, and show progress while it works.
NFR-02	The system shall support at least 5 simultaneous users in the prototype.
NFR-03	Frequently used geographic results (geocoding, routes, POIs) shall be cached to avoid repeated external requests.
NFR-04	External API calls shall be minimized and shall respect each provider's rate limits.

3.2 Security Requirements

ID	Requirement
NFR-05	User data stored in the database shall be protected from unauthorized access.
NFR-06	API credentials shall be kept on the backend and never exposed in the frontend.
NFR-07	The backend shall validate all incoming user data.
NFR-08	Secure communication (HTTPS) shall be used in deployment.
NFR-09	The system shall keep logs of agent runs and external-service errors.

3.3 Software System Attributes

Reliability and accuracy

ID	Requirement
NFR-10	The system shall handle external API failures gracefully, and one failing service shall not stop the whole planning process.
NFR-11	The system shall clearly mark unavailable or incomplete information and shall not present unavailable data as verified.
NFR-12	Factual data (prices, schedules, ratings, safety) shall come from databases or external sources, not only from LLM text.
NFR-13	The system shall distinguish verified information from estimated information.

Usability

ID	Requirement
NFR-14	The interface shall be simple, and results shall be shown in a structured, readable format.
NFR-15	The itinerary shall be readable on desktop and mobile-sized screens.

Scalability, portability and maintainability

ID	Requirement
NFR-16	New agents, data providers and destinations shall be addable without redesigning the system.
NFR-17	The system shall run on Windows and Linux, and in modern web browsers.

ID	Requirement
NFR-18	Agents shall be separate logical components, with tools kept separate from agent logic and database code separate from presentation logic.
NFR-19	The source code shall be managed with Git/GitHub.

Availability

Not Applicable. The prototype is not intended for continuous production service.

4 External Interface Requirements

4.1 User Interfaces

- Travel request page: a form for origin, destination, duration, travelers, budget, preferences and interests, plus a free-text box for natural-language requests.
- Result page: day-wise itinerary, cost breakdown with total and remaining budget, weather and safety notes, and an interactive map with markers and routes.
- Plan page: option to modify a requirement and regenerate the plan.
- Responsive layout with Tailwind CSS; messages show when data is missing or estimated. Wireframes will be added during design.

4.2 Hardware Interfaces

No special hardware interfaces. The backend host needs enough RAM (and preferably a GPU) to run the local LLM through Ollama. Users need a device with a modern web browser.

4.3 Software Interfaces

Software	Version	Source	Purpose
React.js, Tailwind CSS, MapLibre GL JS	Latest stable	Official sites / npm	Frontend, styling, maps
Python, FastAPI	Latest stable	python.org / PyPI	Backend API and logic
LangGraph	Latest stable	PyPI	Agent workflow and state
Ollama + open-source LLM	Latest stable	ollama.com	Local language model
PostgreSQL + PostGIS	Latest stable	postgresql.org / postgis.net	Application and spatial data
Nominatim, Overpass API	Public service	OpenStreetMap	Geocoding and POI discovery
OSRM	Public service	project-osrm.org	Road routes, distance and time
Open-Meteo	Public API	open-meteo.com	Weather data
GTFS / OpenTripPlanner	Where available	Transit agencies	Public transit data

The backend calls these services over REST/JSON. Message formats are defined in each provider's documentation, so only references are given here.

4.4 Communication Interfaces

- Frontend to backend: HTTPS with REST APIs and JSON.
- Backend to external services: HTTPS REST APIs.
- Backend to Ollama and PostgreSQL: local network connections on the host or Docker network.

5 Acceptance Criteria

The software is accepted when the following checks pass:

No.	Check	Method
AC-01	A natural-language request is converted into correct parameters (origin, destination, duration, travelers, budget).	Test with sample requests
AC-02	All four agents run and return structured results to the Supervisor.	Integration test
AC-03	The itinerary is day-wise, includes costs, and the total is compared with the budget.	Scenario test (Kochi to Hyderabad)
AC-04	When the budget is exceeded, the system proposes lower-cost options or a revised plan.	Budget-violation test
AC-05	A simulated heavy-rain warning triggers replanning with a valid alternative.	Replanning test
AC-06	Map shows locations and routes consistent with the itinerary.	UI test
AC-07	When an external service is down, the system still returns a plan and marks missing data.	Failure test
AC-08	Estimated values are labeled as estimates, and no feature claims live booking or verified fares.	Review
AC-09	Each feature is classified as implemented, partially implemented or planned in the demonstration.	Review

6 Appendices

6.1 User Need Document

Agentic AI-Based Intelligent Travel Planning System: Professional Project Summary (Reference 1).

6.2 Requirement Traceability Matrix

Need (Project Summary)	Requirements	Acceptance
Natural-language requirement analysis	FR-01 to FR-12	AC-01
Multi-agent task coordination	FR-13 to FR-22	AC-02
Budget estimation and validation	FR-40 to FR-43	AC-03, AC-04
Transportation and route planning	FR-23 to FR-27	AC-02
Accommodation recommendations	FR-28 to FR-30	AC-02
Geospatial discovery and maps	FR-31 to FR-35, FR-51, FR-52	AC-06
Weather and essential-service awareness	FR-36 to FR-39	AC-05
Day-wise itinerary generation	FR-44 to FR-46	AC-03
Dynamic replanning	FR-47 to FR-50	AC-05

6.3 Product Backlog

Not Applicable.

6.4 Future Enhancements

Real-time transport information, commercial hotel providers and live availability, flight-price integration, saved trips and user accounts, voice and multilingual requests, mobile app, advanced route optimization, real-time alerts and human approval before major itinerary changes.